



DEVELOPER ENABLEMENT · INTERNAL

MVP1 Rohan

Implementation Blueprint

What the build pack contains, why each artifact exists, how to implement it, and how to prove MVP1 works before release.

Stack integrity, orchestrated.

START HERE Read this document before opening individual workbooks. The JSON contracts and blueprint files are authoritative for code; the Excel workbooks explain the business and operating logic around them.

Document	Value
Audience	Rohan and the MVP1 backend implementation team
Prepared by	Klints
Confidentiality	Internal
Pack	Klints_MVP1_Rohan_Build_Pack_v1.2_20260718
Blueprint version	1.2 · 18 July 2026
Current release state	BUILD-READY SPECIFICATION CANDIDATE
Static QA	PASS — 44 workbook sheets rendered; no formula-error tokens
Remaining external gates	Independent runtime validation, live Manago MCP discovery, backend end-to-end dry run

Confidential working documentation. Do not treat a DISCOVERY_REQUIRED capability as live without stored evidence.

How to use this blueprint

This document is the operational front door to the package. It is intentionally more prescriptive than the source workbooks: it tells you which file is authoritative, which object to implement first, what not to infer, which tests must pass, and what evidence is required for release sign-off.

NON-NEGOTIABLE If a field, status, score, capability or workflow step is ambiguous in code, stop and raise a specification issue. Do not silently choose a meaning or add a platform assumption.

Recommended reading path

1. Read Sections 1–4 to understand the product boundary and artifact precedence.
2. Implement the 13 JSON Schemas and version registry before any business logic.
3. Implement the exact 42-check DCS registry and reproduce the Lumera golden score.
4. Implement capability resolution and record the result of Manago MCP discovery.
5. Implement the architecture graph, pilot registry and deterministic task orchestrator.
6. Generate Manago build packages from the 16 blueprint JSON files.
7. Run the complete test and release-gate sequence in Sections 12–15.

Release truth at a glance

Area	Pack status	What this means
DCS scope	LOCKED	42 headline checks plus 12 on-demand pilot preflight gates; supplemental gates do not alter headline score
Pilot scope	LOCKED	Exactly 16 authoritative workflow blueprint JSON files
Scoring	SPECIFIED	0–100 model, gates, coverage, confidence and worked fixture supplied
Manago REST / Agent	DOCUMENTED	Use only operations marked CONFIRMED_LIVE or CONFIRMED_LIMITED
Manago MCP	DISCOVERY REQUIRED	New channel anticipated by design but not proven by supplied Manago documentation
Workflow activation	APPROVAL-GATED	No customer-data write, workflow publish or activation without a valid approval token
Production proof	PENDING	Requires Rohan runtime tests, live platform evidence and end-to-end sign-off
Assessment Report	SPEC_VALIDATED	REPORT/EXPORT, schema, API, security, fixtures and acceptance tests supplied; backend NOT_RUN

1. Product intent and implementation boundary

Klints is the neutral architecture and integrity layer above an ecommerce brand's existing stack. It connects CDP/marketing automation, commerce, ERP, analytics and paid-media data; determines whether the data is reliable enough for lifecycle activity; identifies architecture gaps; prepares governed fixes and workflow build guides; validates readiness; and emits machine- and human-executable handoff packages.

1.1 The core invariant

THE SCORE GATES THE BRAIN Klints must not recommend or activate lifecycle orchestration on top of broken data. A headline DCS above the tenant threshold is necessary but not sufficient: every workflow-specific gating check must also pass or carry an explicitly approved WARN.

1.2 What MVP1 must do

- Ingest and normalize evidence from the controlled connector set.
- Run exactly 42 MVP1 Data Consistency checks and produce an explainable headline score plus seven sub-scores.
- Map existing platform assets and dependencies without issuing unsafe destructive verdicts.
- Select and prepare exactly 16 pilot use cases using versioned, machine-readable blueprints.
- Resolve each execution step to a confirmed Manago channel or an approved fallback.
- Sequence work deterministically into dependency-safe waves.
- Require approval before side effects, run QA before staging, and emit an auditable handoff package.

1.3 What MVP1 must not assume

- That Manago MCP can read or write a particular object because a workbook anticipates the capability.
- That the Manago Agent can build or modify workflow graphs; supplied documentation says it cannot today.
- That a partial connector sweep can produce a final score.
- That NOT_CONNECTED is equivalent to FAIL, or that missing evidence may be silently excluded.
- That a high headline DCS overrides a failed consent, identity or workflow-specific gate.
- That a workflow can be retired or consolidated without complete where-used/dependency evidence.
- That agent-generated content or segments are approved production objects rather than drafts requiring governance.

1.4 Definition of done

MVP1 is functionally proven only when the same immutable inputs produce the expected DCS, architecture verdict, pilot readiness, task order, execution path, QA outcome and handoff package; all side effects are approval-gated and idempotent; and all capabilities used in a live tenant are supported by stored evidence or an approved fallback.

2. What the build pack contains

The package is organized by authority and implementation responsibility. Do not flatten the folder structure; relative paths are referenced by manifests, acceptance tests and this blueprint.

Folder	Purpose	How Rohan should use it
01_Specifications	Four v1.4.1 product/logic workbooks	Read for rules, rationale, thresholds, examples and human-operating context
02_Execution_Capabilities	Channel-level truth for Manago REST, Agent, SDK, callbacks, MCP and human fallback	Implement the capability resolver; persist verification evidence and fallback decisions
03_Machine_Contracts	13 JSON Schemas	Load into a versioned schema registry; validate every persisted/emitted object
04_MVP1_Pilot_Blueprints	Sixteen authoritative build-guide JSON files plus pilot manifest	Treat JSON as source of truth for workflow builds and handoff
05_Test_Fixtures	Lumera golden tenant, expected outputs and edge cases	Use as deterministic regression and release-blocking tests
06_Implementation	README, backlog, acceptance matrix and this blueprint	Use as implementation sequence and sign-off record
manifest.json	Version and SHA-256 inventory	Verify package integrity before starting and at handoff
qa_report.json	Workbook render and formula-error scan report	Evidence of static workbook QA; not proof of platform behavior

2.1 Specification workbooks

Workbook	Authoritative additions in v1.4.1	Backend output
InitialDataConsistencyCheck	Scoring model, score assembly, exact 42-check scope, Lumera scoring example	check definitions/results, findings, dimension scores, headline score and workflow gates
ArchitectureAssessmentFramework	Dependency rules, architecture verdict, 16-stage lifecycle model, capability requirements	asset graph, per-asset verdicts, architecture mode and evidence coverage
DefaultUseCaseLibrary	Capability map, learning loop, vertical overlays, exact 16 pilots, build-guide contract	pilot registry, readiness decision and workflow blueprint selection

Workbook	Authoritative additions in v1.4.1	Backend output
OnboardingOrchestrationBlueprint	Lumera tasks, canonical task schema/algorithm, REPORT/EXPORT and Assessment Report contract	validated DAG, canonical priority, governed report payload, PDF/JSON artifacts, waves, approvals and handoff

2.2 Machine contracts

Schema	Object	Implementation responsibility
check_definition.schema.json	Versioned definition of a DCS check	Registry keyed by check_id + catalog version
check_result.schema.json	Outcome and evidence for one check/run	Append-only result with status, confidence, evidence and scoring model version
finding.schema.json	Actionable issue produced by a failed/warned check	Lifecycle state, blockers, root causes and remediation
architecture_asset.schema.json	Workflow/segment/tag/property/surface/etc.	Normalized asset inventory and dependency edges
architecture_verdict.schema.json	Assessment output	Mode plus evidence-backed per-asset verdicts
workflow_blueprint.schema.json	Authoritative pilot build contract	Workflow graph, gates, measurement, approvals, rollback and capability dependencies
orchestration_task.schema.json	One executable/approvable task	State machine, dependencies, priority, wave, capabilities, provenance and idempotency
dc_run.schema.json	Aggregate DCS run and headline state	Coverage, confidence, dimensions, score/null and check-result lineage
capability_record.schema.json	Versioned operation capability	Channel, mode, evidence, status and deterministic fallback
approval_token.schema.json	Scoped human approval	Actor, tenant, object version, diff hash, expiry and status
qa_result.schema.json	Workflow/report QA outcome	Score plus release-blocking hard-test evidence
handoff_package.schema.json	Signed staged handoff	Artifact, QA, approval and manifest-hash references
assessment_report.schema.json	Governed downloadable report	Variant, content gate, lineage, access policy, artifacts and hashes

2.3 Artifact precedence

WHEN FILES APPEAR TO DISAGREE Use this order: JSON Schema → authoritative blueprint JSON / pilot manifest → v1.4.1 workbook contract sheets → acceptance matrix → README → narrative source material. Raise an issue if the higher-authority artifact is incomplete; do not patch meaning in a lower layer only.

3. How the artifacts compose

The package is one system contract, not a collection of independent documents. Every downstream decision must retain provenance back to the evidence and version that produced it.

Stage	Consumes	Produces	Hard gate
1 · Connect	Tenant credentials, scopes, topology	Connector state and capability evidence	Required connectors usable; partial sweep state explicit
2 · Normalize	Manago, commerce and ERP data	Canonical contacts, events, products, consent and business truth	Identity keys, dedupe and source precedence applied
3 · Score	Normalized evidence + 42 definitions	Check results, findings, seven sub-scores and headline DCS	Foundation gates and ≥80% required-dimension coverage
4 · Assess	DCS + platform asset inventory	Dependency graph, per-asset verdicts and architecture mode	No Retire/Consolidate without where-used proof
5 · Select	Architecture gaps + DCS gates + 16-pilot manifest	Ready/blocked pilot set	DCS ≥70 and all pilot-specific checks PASS/approved WARN
6 · Orchestrate	Findings, verdicts, blueprints and capabilities	Deterministic tasks, priorities and waves	DAG valid; capability live or fallback approved
7 · Build & QA	Blueprint + resolved executor + approvals	Staged workflow/config and QA result	QA ≥80 and every hard test passes
8 · Handoff	Approved, validated versioned outputs	MCP action object / A2A task spec / human guide	Signed audit, idempotency key, no autonomous activation

3.1 Required provenance chain

At minimum, each recommendation and task must point to the tenant, connector snapshot/version, check catalog, scoring model, architecture assessment, blueprint version, capability evidence and approval record. Replaying an old decision under a new scoring or blueprint version creates a new record; historical results are never silently recomputed in place.

3.2 Minimum persisted identifiers

- tenant_id and account/sub-account scope
- connector_run_id and source timestamps
- check_catalog_version and scoring_model_version

- check_result IDs and finding IDs
- assessment_id and asset/version IDs
- blueprint_id, use_case_id, variant_id and blueprint version
- capability IDs plus verified_at/evidence or fallback approval
- task_id, idempotency_key, approval token/diff hash and audit events

COMMON-MODE FAILURE TO AVOID Do not let the same implementation function both calculate a result and generate its expected test value. Golden expected outputs must remain immutable and independently checked.

4. Data Consistency Score implementation

The DCS establishes whether the stack is safe enough to support lifecycle activity. The full catalogue contains 102 checks for expansion; MVP1 implementation scope is exactly the 42 IDs listed in workbook sheet '09 MVP1 Check Scope' and Appendix B of this document.

4.1 Score dimensions and weights

Dimension	Headline weight	Required	Purpose
Customer Identity	18%	Yes	Customer-universe integrity, duplicates and identity match
Lifecycle Event	18%	Yes	Purchase/cart/return completeness and timeliness
Product & Transaction	14%	Yes	Product resolution and transaction truth
Segment & Property	12%	Yes	Decision-field schema, namespace and population sanity
Channel & Consent	18%	Yes	Channel consent truth and propagation
Measurement	10%	Yes	Attribution wiring and baseline computability
Business Reality	10%	Conditional	ERP margin/inventory truth; optional only by declared tenant scope

4.2 Per-check scoring

Status	Factor	Treatment
PASS	1.0	Evaluated and meets threshold
WARN	0.5	Evaluated with partial degradation; approval may still be required
FAIL	0.0	Evaluated and fails threshold

Status	Factor	Treatment
NOT_APPLICABLE	Excluded	Must carry a defensible reason/evidence
NOT_CONNECTED	Excluded conditionally	Only permitted for a formally out-of-scope optional dimension
UNKNOWN	Excluded but lowers coverage	Missing/unreliable evidence; may force INCOMPLETE

Numeric check weights are Critical=5, High=4, Medium=2, Low=1 and Informational=0.5. Foundation Gates carry numeric weight 0 because they determine whether scoring is permitted, not how much an issue pulls the score.

4.3 Exact formulas

DIMENSION SCORE $100 \times \Sigma(\text{numeric_weight} \times \text{result_factor}) \div \Sigma(\text{eligible_numeric_weight})$. Eligible means PASS, WARN or FAIL only. Store at least four decimals and round only for display.

COVERAGE $\Sigma(\text{eligible_numeric_weight}) \div \Sigma(\text{scoped_applicable_numeric_weight})$. Required dimensions need at least 80% coverage before a final headline score may be published.

CONFIDENCE $\Sigma(\text{eligible_numeric_weight} \times \text{confidence_factor}) \div \Sigma(\text{eligible_numeric_weight})$, where HIGH=1.0, MEDIUM=0.7 and LOW=0.4. Confidence does not alter the score; it alters recommendation status and audit presentation.

HEADLINE $\Sigma(\text{dimension_score} \times \text{dimension_weight}) \div \Sigma(\text{included_dimension_weight})$. Business Reality may be excluded only when ERP is formally outside tenant scope; the state is then capped at CONDITIONALLY_READY and confidence at 85%.

4.4 Final states

State	Condition	System behavior
BLOCKED	Blocking Foundation Gate fails or DCS <50	No workflow recommendation or activation
INCOMPLETE	Required coverage <80% or sweep incomplete	Only provisional sub-scores; no final DCS
REMEDiate	DCS 50–69.99	Data fixes first; workflow builds held
CONDITIONALLY_READY	DCS 70–89.99	Only workflows with satisfied specific gates
READY	DCS ≥90	May proceed subject to workflow gates, capabilities, approvals and QA

4.5 Golden result

RELEASE-BLOCKING EXPECTATION Running the supplied Lumera check_results.json through an independent implementation must produce headline_score = 84.2670 and score_state = CONDITIONALLY_READY. A different result is a P0 defect until explained and approved as a versioned model change.

5. Architecture assessment implementation

The assessment determines what already exists, what is safe, what should be improved, and whether Klints should augment or selectively rebuild the tenant architecture. It is evidence-based and recommendation-only; the operator decides on destructive actions.

5.1 Asset model

Normalize workflows, segments, tags, properties, surfaces, recommendation strategies, channels and metrics into architecture_asset objects. Preserve platform IDs, versions/status, complete definitions where available, and capability/evidence provenance.

5.2 Dependency graph safety

- Workflow USES segment.
- Workflow READS tag/property and SENDS through a channel.
- Surface TARGETS a segment.
- Recommendation READS catalog fields.
- Metric ATTRIBUTES results to workflow.
- Use case is GATED_BY DCS checks.
- Tasks DEPEND_ON tasks; workflows may COLLIDE_WITH workflows; consolidation REPLACES an asset.

DESTRUCTIVE-VERDICT GATE Any incoming dependency edge disqualifies an asset from Retire candidate. Consolidate requires proof that the canonical survivor absorbs every dependent and an approved migration plan.

5.3 Per-asset verdict order

1. Evaluate compliance and Fix first before all other verdicts.
2. Evaluate the dependency gate before Consolidate or Retire candidate.
3. Evaluate redundancy before inactivity: duplicates consolidate; they are not simply retired.
4. Use Retire candidate only for dependency-free, duplicate-free and value-free assets.

5. Use Keep or Keep + Improve for surviving assets.

5.4 Architecture-level modes

Mode	Rule	Use-case behavior
AUGMENT	Weighted architecture score ≥ 70 , fewer than two critical defects and no unmitigated consent breach	Preserve proven fixed points; propose only gaps and improvements
SELECTIVE_REBUILD	Score 50–69.99 or one bounded critical cluster	Rebuild affected lifecycle groups; preserve unaffected assets
REBUILD	Score < 50 , at least two critical defects, or architecture is structurally irrecoverable	Propose a coherent fresh architecture and carry forward salvageable Keep assets
INCOMPLETE	Required evidence/dependency coverage missing	Emit discovery tasks; issue no destructive verdict

5.5 Capability-sensitive evidence

The supplied REST v2 workflow list returns ID and name, not the full trigger/condition/action graph. Therefore workflow-definition reads, reverse where-used analysis and surface introspection must resolve through confirmed MCP operations, a human/export fallback, or an INCOMPLETE result. Never synthesize a complete graph from names and aggregate statistics.

6. MVP1 use cases and workflow blueprints

The catalogue remains a broad learning library, while MVP1 build scope is fixed by pilot_manifest.json. Exactly 16 JSON files are authoritative for backend ingestion and Manago build generation.

Rank	UC	Pilot	Lifecycle area
1	UC-02	Welcome series continuity	Acquisition
2	UC-04	First-purchase incentive delivery	Acquisition
3	UC-05	Double opt-in completion nudge	Acquisition
4	UC-06B	Second Purchase Accelerator — omnichannel flagship	Second purchase
5	UC-08	Post-purchase education / product usage	Onboarding
6	UC-09	Review & UGC request	Onboarding
7	UC-10	Account creation nudge	Onboarding
8	UC-11	Replenishment reminder	Repeat
9	UC-12	Back-in-stock notification	Repeat
10	UC-13	Price-drop alert	Repeat
11	UC-16	Churn-risk save	Retention
12	UC-17	Winback lapsed customers	Retention

Rank	UC	Pilot	Lifecycle area
13	UC-21	Cart abandonment recovery	Recovery
14	UC-23	VIP candidate progression	Loyalty
15	UC-28	Complementary product cross-sell	Expansion
16	UC-36	Post-purchase upsell to premium tier	Expansion

6.1 Resolved UC-06 split

UC-06A Simple single-channel second-purchase email nudge. Retained as a non-pilot fallback variant for tenants that intentionally downgrade the flagship orchestration.

UC-06B The pilot flagship: persistent holdout, email, consent-aware SMS/Web Push escalation, global purchase exit, seven-day on-site recommendation eligibility and channel economics.

6.2 Why UC-41 is outside the exact 16

UC-41 depends on programmatic on-site surface creation. The package anticipates MCP.SURFACE.UPSERT, but the supplied Manago documentation does not confirm it and functional parity with a human UI build has not been proven. UC-41 remains in discovery backlog and must not become a core pilot by assumption.

6.3 Required blueprint sections

- Identity/version: stable `blueprint_id`, `use_case_id`, `variant_id` and release.
- Data gates: minimum DCS, required check IDs, events, fields and freshness.
- Audience/consent: inclusions, exclusions and per-channel consent evaluated at send time.
- Workflow graph: unique node IDs, node type, platform primitive, config, `configured_by`, `executed_by` and next edges.
- Pressure controls: suppressions, frequency policy, collision policy and precedence.
- Measurement: KPI, attribution window, holdout and net-impact logic.
- QA/approval/rollback: hard tests, approval binding, reversible behavior and audit provenance.
- Capability dependencies and handoff: preferred path, approved fallback and idempotency key.

6.4 Namespace convention

DO NOT MIX NAMESPACES Contact details/properties use the `klints_` prefix. Tags use the `klints:` prefix. `configured_by` says who prepared the node/config; `executed_by` says which system actually runs it.

6.5 Graph validation

Before a blueprint can enter BUILD, validate that every node ID is unique, every next target exists, all non-terminal paths reach a terminal, no orphan nodes exist, cycles are either prohibited or explicitly modeled, consent and suppression guards precede every marketing send, and a global conversion exit prevents post-purchase messages from continuing.

7. Manago execution capability and MCP

Execution must be capability-driven rather than hard-coded to one integration channel. The same workflow blueprint can remain stable while a node's executor changes from Human to MCP or Agent after verified capability evidence.

7.1 Execution channels

Channel	Use	MVP1 rule
MANAGO_AGENT	Draft emails, create segments, define recommendations, targeted analytics	Treat output as draft; Klints validates and human approves
REST_V2 / REST_V3	Documented contacts/events/catalog/workflow-list/stats/channel operations	Respect paging, batch caps, windows, throttling and source limitations
FRONTEND_SDK	Identity and behavioral event tracking	Verify storefront installation and identity continuity
CALLBACK	Delivery/engagement notification where supported	Use polling fallback when callback coverage is insufficient
MANAGO_MCP	New tool channel for reads/writes if confirmed	Every operation begins DISCOVERY_REQUIRED; evidence is mandatory
HUMAN_OPERATOR	Workflow build, publish/activation, approvals and unsupported surfaces	Confirmed MVP1 fallback, not an error state

7.2 Capability statuses

Status	Meaning	Resolver behavior
CONFIRMED_LIVE	Operation proven and supported for defined scope	May route when tenant scopes and approvals allow
CONFIRMED_LIMITED	Operation exists with documented limitation	Route only within recorded caps; otherwise fallback
DISCOVERY_REQUIRED	Design anticipates operation; no sufficient execution evidence	Do not execute; use approved fallback or block
NOT_SUPPORTED	Current channel cannot perform operation	Use fallback; never retry as capability discovery

Status	Meaning	Resolver behavior
DEPRECATED	Legacy path should not receive new work	Route to replacement and preserve migration audit

7.3 MCP discovery sequence

1. Authenticate to the tenant MCP endpoint and record the exact tool catalog, schemas, granted scopes and server/version identifiers.
2. Run read-only tests first: workflow list/graph/stats, segments, where-used, tags/properties, surfaces, analytics, pagination/resume and structured errors.
3. Store request hash, sanitized response artifact, observed limits, verified_at, environment and verifier for every test.
4. Run workflow/surface write tests only in a disabled sandbox/draft and only after explicit approval.
5. Prove idempotent create/update behavior and restore the prior version; never activate production during discovery.
6. Update capability status at operation level, not globally for the MCP channel.

7.4 Resolver pseudologic

CAPABILITY RESOLUTION If preferred capability is CONFIRMED_LIVE and tenant scope permits it, route there. Else if an approved fallback exists, route to the fallback and persist why. Else return BLOCKED_CAPABILITY. DISCOVERY_REQUIRED is never equivalent to success.

7.5 MVP1 fallback guarantee

Every core pilot can produce a complete, platform-specific human build guide if MCP workflow upsert is not confirmed. No autonomous publish is required for MVP1. Human activation remains the safest default even after draft-write capability is discovered.

8. Orchestration engine

The orchestrator transforms connector, DCS, architecture, blueprint and capability outputs into a deterministic task plan. It must be reproducible: the same input graph and values produce the same priority order and waves.

8.1 Task state machine

From	To	Allowed when
PENDING	BLOCKED	Dependency, capability or gate is unmet

From	To	Allowed when
PENDING / BLOCKED	READY	All hard conditions are met
READY	IN_PROGRESS	Executor lease acquired
IN_PROGRESS	AWAITING_APPROVAL	A side-effect proposal/diff is ready
AWAITING_APPROVAL	IN_PROGRESS	Valid approval token matches tenant, object, version and diff hash
IN_PROGRESS	DONE	Acceptance checks pass and outputs persist
IN_PROGRESS	FAILED	Error and retry class persist; rollback executed if needed
Any non-terminal	CANCELLED	Authorized cancellation and partial rollback

8.2 Priority formula

PRIORITY SCORE $0.40 \times \text{blocker_status} + 0.30 \times \text{severity_risk} + 0.20 \times \text{dependency_readiness} + 0.10 \times \text{effort_impact}$. Every input is an integer from 0 to 3 and the score range is 0 to 3, matching the canonical v1.3 formula and worked examples. P0 hard gates still take precedence over numeric score.

8.3 Deterministic dependency scheduling

1. Validate that every dependency ID exists and that no task depends on itself.
2. Detect strongly connected components. Any SCC with more than one node, or a self-loop, is **BLOCKED_DEPENDENCY_CYCLE** and must include the cycle path.
3. Use Kahn topological ordering for the validated DAG.
4. Break ties by priority class P0→P2, priority score descending, unblock count descending, effort ascending, then task_id lexicographically.
5. Assign wave as the greater of task-type floor and one plus the maximum prerequisite wave.

8.4 Wave floors

Wave	Content	Exit gate
0	CONNECT / DISCOVER	Credentials, topology and capability state recorded
1	SCORE	DCS complete or explicit INCOMPLETE
2	FIX / ASSESS	P0 approvals resolved and dependency evidence sufficient
3	REPORT / EXPORT / PLAN	Diagnostic report validates and selected-pilot gates pass

Wave	Content	Exit gate
4	BUILD	Execution capability or fallback resolved
5	QA	QA ≥80 and all hard tests PASS
6	REPORT / EXPORT / HANDOFF	Final artifacts hashed; approval, audit and signed package complete

8.5 Side-effect controls

- Every customer-data or platform write requires an idempotency key and request hash.
- Approval binds tenant, target object, version and exact diff; a generic standing approval is insufficient.
- The executor must not approve its own side effect.
- Retries must classify transient vs permanent errors and must not duplicate writes.
- A partial failure triggers documented rollback and preserves the evidence chain.

8.6 Assessment Report and export pipeline

The downloadable Assessment Report is a release-blocking product surface and a first-class orchestration output. Do not implement it as an ad-hoc print action from the UI. REPORT composes one governed payload from immutable snapshots; EXPORT renders PDF and JSON artifacts from that exact payload.

Diagnostic sequence

- After DCS and Architecture Assessment complete, create REPORT-DIAG with the FREE_DIAGNOSTIC variant.
- Validate assessment_report.schema.json before rendering. If required inputs are missing, emit INCOMPLETE with headline_score = null and explicit coverage/missing-input disclosure.
- Create EXPORT-DIAG only from the validated payload. PDF and JSON must reference the same report version, template version and payload hash.
- After paid plan/QA work, create a new PAID_FULL or LIVING report version. Never overwrite the earlier diagnostic artifact.

Content boundary

- FREE_DIAGNOSTIC includes the complete DCS diagnostic, issue counts, Architecture verdict, top three to five issues and business-impact estimate; remediation, workflows, execution plan, writeback and monitoring remain locked.
- PAID_FULL adds check-level detail, root causes, fix/workflow blueprints, prioritized plan and remediation/QA status.
- Default report payloads contain aggregates only. Contact-level rows, emails, phones, contact IDs and raw events must not appear in PDF or JSON.

API, storage and security

- Implement create, status and download operations exactly as defined in `assessment_report_api.openapi.json`. Creation and regeneration require an Idempotency-Key.
- Authorize tenant scope before returning metadata, status or bytes. Cross-tenant requests must fail without leaking object existence. Audit every successful and denied download.
- Persist template version, input snapshot IDs, payload hash, per-file SHA-256, storage region and `retention_policy_id`. A retry with the same key and payload returns the prior job/result; a different payload conflicts.
- Renderer or storage failure removes/invalidates partial artifacts and leaves the report FAILED with a retry class. An artifact is downloadable only in READY state.

9. Recommended implementation sequence

Build in vertical slices that preserve the contract chain. Do not start by coding all connectors or all workflows independently; first prove that one complete path can move from evidence to an approved handoff.

Milestone A — contracts, connectors and scoring

- Load and version all 13 schemas; reject invalid or unknown schema versions.
- Implement canonical tenant/run/provenance storage and immutable fixture loading.
- Implement connector checkpoint/resume and explicit incomplete-sweep state.
- Register exactly 42 check definitions and scoring weights.
- Reproduce Lumera DCS and all scoring edge cases independently.
- Persist capability definitions and resolver behavior.

Milestone B — architecture, pilots and orchestration

- Normalize architecture assets and dependency edges.
- Implement per-asset verdict order and architecture mode with evidence coverage.
- Load `pilot_manifest.json` and validate exactly 16 unique blueprints.
- Join blueprint gating_check_ids to the latest compatible DCS run.
- Implement task state machine, priority formula, DAG validation, cycle detection and waves.

Milestone C — build packages, approval, QA and handoff

- Generate a complete Manago-specific build guide from blueprint JSON.
- Resolve preferred capability vs fallback and persist the decision.
- Implement diff-bound approval tokens and audit events.
- Implement QA score ≥ 80 plus hard-fail domains.

- Emit staged-not-live MCP/A2A or human package with idempotency and rollback metadata.
- Pass Lumera end to end and freeze a release manifest.

9.1 Required first vertical slices

Slice	Purpose	Expected proof
DCS slice	Load 42 results and score	84.2670 / CONDITIONALLY_READY plus exact seven sub-scores
UC-02 slice	Simple workflow build	Gates → task plan → human/MCP-resolved guide → approval → QA
UC-06B slice	Complex orchestration	Holdout, consent branching, purchase exit, collision controls and fallback
Lumera E2E	Whole package contract	Connector snapshot → DCS → assessment → readiness → waves → QA → handoff

IMPLEMENTATION FEEDBACK RULE When the codebase reveals a specification gap, record a P0/P1/P2 issue with exact artifact, version, field/path, conflicting interpretations, proposed resolution and impacted fixtures. Do not fix only in code.

10. Test strategy

The release must be tested in layers. Static file validity is necessary but does not prove scoring correctness, workflow safety or Manago behavior. Each layer below has an explicit output and release consequence.

10.1 Package integrity tests

- Verify the ZIP extracts without error and preserves the expected folder structure.
- Recompute SHA-256 and byte length for every manifest entry.
- Reject missing, extra-unapproved or mismatched files.
- Assert exactly 13 schemas, sixteen UC blueprint JSON files, forty-two Lumera check results and one pilot manifest.
- Parse every JSON file before running semantic tests.

10.2 Schema and referential-integrity tests

- Validate schemas with a standards-compliant JSON Schema Draft 2020-12 validator in Rohan's runtime.
- Validate all blueprint and fixture objects; reject additional properties where the schema forbids them.
- Assert unique IDs and supported schema versions.

- Resolve every gating check ID, capability ID, dependency task ID, workflow edge target and manifest file path.
- Assert pilot_manifest count=16 and ranks 1–16 without gaps.
- Assert details use klints_ and tags use klints:.

10.3 DCS tests

Test	Fixture	Expected
Golden Lumera	lumera_expected_results/check_results.json	84.2670 and CONDITIONALLY_READY
All pass	edge_cases/all_pass.json	100.0000 and READY
Foundation gate fail	edge_cases/gate_fail.json	BLOCKED; no headline score
ERP excluded	edge_cases/erp_not_connected.json	Business Reality excluded only with reason; state/confidence capped
Partial sweep	edge_cases/partial_sweep.json	INCOMPLETE; no final headline
Consent failures	edge_cases/consent_fail.json	Relevant workflows blocked regardless of headline

10.4 Architecture tests

- An asset with a dependent workflow cannot receive Retire candidate.
- Consolidation without a complete migration target becomes INCOMPLETE or remains Keep.
- A DCS-linked harmful asset receives Fix first before any usage-based verdict.
- Missing workflow graph evidence prevents destructive verdicts.
- Overall mode is deterministic from weighted inputs, critical defects and evidence coverage.

10.5 Blueprint graph and safety tests

- Unique nodes, valid edges, reachable terminal and no undeclared cycles/orphans.
- Every marketing channel action is preceded by its consent guard.
- Global suppressions and frequency policy are evaluated at execution time.
- Conversion/purchase exits stop all remaining branches.
- UC-06B and UC-16 retain persistent holdouts for the measurement window.
- No action executes when the required capability is unconfirmed and no fallback is approved.
- Rollback is possible for every created workflow version and klints_ write.

10.6 Orchestration tests

- Priority scores match the formula to an agreed tolerance and cannot be manually overwritten.

- Topological order is stable across repeated runs and machine instances.
- Tie-breaks follow the exact deterministic order.
- `dependency_cycle.json` emits `BLOCKED_DEPENDENCY_CYCLE` and the exact cycle path.
- No task enters `READY` before all hard dependencies, data gates and capability rules are satisfied.
- Approval-token mismatch/expiry returns `AWAITING_APPROVAL` without a side effect.
- Idempotent replay returns the prior result; the same key with a different payload returns conflict.

10.7 Assessment Report tests

- Validate report schema and OpenAPI documents with standard parsers.
- Verify `FREE_DIAGNOSTIC` includes required diagnostic sections and exposes no remediation content.
- Verify an incomplete sweep produces `INCOMPLETE`, `headline_score` = null and visible missing-input/coverage disclosure.
- Verify PDF and JSON derive from the same payload hash and immutable version lineage.
- Run cross-tenant, expired-download, PII scan, retry/idempotency and partial-render cleanup tests.

11. Live Manago and connector verification

Live verification should use a dedicated sandbox or disabled draft objects where possible. Evidence must be tenant- and environment-specific; a capability proven in one tenant is not automatically granted in another if scopes or product configuration differ.

11.1 Connector checks

System	Minimum verification	Failure behavior
Manago	Authentication, account scope, contact/event/product reads, workflow list/stats, documented caps	Block or mark affected assessment incomplete; never fabricate full graph
Commerce	Customer/order/product/inventory scopes, pagination, historical window and timestamps	Block affected checks; no partial final score
ERP	Feed reachability, schema mapping, row count, encoding, margin/inventory freshness	If formally optional, mark Business Reality <code>NOT_CONNECTED</code> with scope reason
Storefront SDK	Tracking script, smclient issuance, identity link and recent <code>VISIT/CART</code> events	Block behavior-dependent use cases

11.2 Live MCP acceptance evidence

A MCP operation may move from `DISCOVERY_REQUIRED` only when the evidence log contains capability ID, exact server/tool/schema version, environment, tenant scope, test ID, sanitized request/response artifacts, paging/limit observations, `verified_at` and verifier. Write operations additionally need approval, disabled/draft target, idempotency proof and rollback evidence.

11.3 Minimum live workflow spike

1. Build or manually reproduce UC-02 in a non-production Manago context.
2. Capture the resulting workflow ID/version and compare the platform graph node-by-node with the blueprint.
3. Confirm entry audience, consent guard, waits, exits, measurement and deactivation/rollback.
4. Repeat the resolver with MCP disabled to prove the human fallback generates a usable guide.
5. Do not activate to real contacts during the discovery spike.

11.4 What can remain pending

A capability that is not required by a core pilot may remain `DISCOVERY_REQUIRED` if the operation is explicitly outside MVP1 and no downstream artifact claims it is available. A capability required by a core pilot may remain pending only when an approved fallback provides the required result and the task/audit record says which path was used.

DO NOT OVERSTATE MCP The design is MCP-ready. The package is not evidence that every Manago MCP read/write operation is currently available. Capability truth comes from executed discovery, not documentation intent.

12. QA, approval and handoff

12.1 QA gate

A workflow needs QA score ≥ 80 and every hard domain must pass. A high average cannot compensate for a failed consent, data-readiness, trigger, suppression, terminal-path, rollback or measurement test.

QA domain	Hard expectations
Data readiness	Latest compatible DCS run; all gating checks PASS or approved WARN; required fields/events populated
Audience	Segment definition reproducible; inclusions/exclusions and tenant scope correct
Trigger and graph	Trigger semantics correct; waits/branches/exits deterministic; terminal reachable
Consent/compliance	Consent checked at send time per channel; opt-outs and erasure respected
Pressure/collision	Frequency cap, active-flow suppression and one-message-per-moment precedence applied
Content/offer	Brand-reviewed draft; valid coupon/offer; product availability and economics protected
Measurement	Primary KPI, attribution, baseline and required holdout wired before staging
Capability/build	Manago primitives and executor path validated; human fallback guide complete if needed
Rollback/audit	Created version can be disabled/restored; all writes and approvals traceable

12.2 Approval token

An approval token must bind tenant_id, task_id or target object, exact proposed version, diff hash, approver identity/role, approved scope, issued_at and expiry. If any bound value changes, the proposal requires a new approval.

12.3 Handoff package

- Validated workflow blueprint and schema version.
- Resolved execution path and capability evidence/fallback decision.
- Platform-specific steps or MCP/A2A payload in STAGED_NOT_LIVE state.
- Approval record, QA result and hard-test outcomes.
- Idempotency key, request/diff hashes and rollback plan.
- Score-at-emit, assessment/version provenance and audit event chain.

12.4 Activation boundary

MVP1 DEFAULT Klints prepares, validates and hands off. Production activation remains a separate human-approved action. Discovery of MCP publish capability does not automatically change this operating boundary.

13. Defect management and specification change

This package will evolve during implementation. Changes are acceptable only when they are explicit, versioned and propagated through all affected artifacts and fixtures.

13.1 Severity

Severity	Definition	Release effect
P0	Could produce wrong score, illegal send, unsafe write, broken dependency order, irreproducible result or unusable core pilot	Block implementation/release until resolved
P1	Ambiguity or gap with a safe workaround/fallback, or material non-core inconsistency	Resolve before affected milestone or accept formally
P2	Editorial, usability or non-blocking extension issue	Backlog; does not block MVP1 if documented

13.2 Issue template

- Issue ID and severity.
- Artifact, file version and exact sheet/cell or JSON path.
- Observed ambiguity/error and two or more possible interpretations.
- Expected product behavior and safety/business impact.
- Affected schemas, fixtures, blueprints, checks, capabilities and tests.
- Proposed resolution and decision owner.
- Regression tests that must change or be added.

13.3 Change-control rule

NO CODE-ONLY SPEC FIXES If implementation requires a contract change, update the authoritative schema/JSON/workbook, increment the appropriate version, update fixtures and manifests, and document migration behavior. Code must not become the only place where the rule exists.

13.4 Compatibility

Persist object schema versions and model/catalog versions. Readers should reject unsupported major versions, tolerate approved compatible minor versions, and preserve old records. A scoring-model change creates a new score record; a blueprint change creates a new blueprint version and may require new approval.

14. Final release gates and sign-off

The package should be called production-verified only after all seven gates below are satisfied or an explicitly named decision owner accepts a documented condition.

Gate	Owner	Pass criterion	Blocking evidence
G1 · Integrity	Rohan	ZIP, manifest and counts match	No checksum or parse errors
G2 · Requirements	Klints Product	Every MVP1 requirement maps to an implemented object/test	No uncontrolled scope changes
G3 · Contracts	Rohan	Standard JSON Schema validator passes all objects	No unknown IDs or schema violations
G4 · DCS	Rohan + Data	Independent golden and edge-case results exact	84.2670 Lumera and all state gates
G5 · Workflow logic	CRM / Manago specialist	All 16 pilot graphs, consent, collisions, measurement and rollback approved	No unresolved P0 logic defects
G6 · Platform	Rohan + Manago	Required operations confirmed or fallback approved	MCP evidence and connector scope

Gate	Owner	Pass criterion	Blocking evidence
G7 · E2E	Rohan + Klints	Lumera vertical path emits approved staged handoff	QA, approval, audit and idempotency proof

14.1 Required signatories

Role	What the sign-off confirms
Klints Product Owner	MVP1 scope, business rules, pilot selection and product behavior
Rohan / Backend Owner	Implementability, schemas, deterministic outputs, tests and operational failure handling
CRM / Manago Specialist	Platform primitives, workflow behavior, channel logic and fallback usability
Data / Governance Owner	Consent, customer-data writes, approvals, audit and rollback

14.2 Current status

Control	Current state
Workbook static/visual QA	SPEC_VALIDATED
Internal JSON parse/contract fixture checks	SPEC_VALIDATED
Exact 42-check and 16-pilot counts	SPEC_VALIDATED · plus 12 supplemental preflight gates
Independent validation in Rohan runtime	NOT_RUN
Live Manago MCP discovery	BLOCKED_EXTERNAL
Backend Lumera end-to-end dry run	NOT_RUN
Final multi-role sign-off	PENDING

FINAL VERDICT TODAY The corrected package is VALIDATED FOR IMPLEMENTATION WITH RESIDUAL EXTERNAL RISKS. It is not yet a production-proven implementation contract until the independent runtime, live capability and end-to-end gates pass.

Appendix A — Exact MVP1 pilot manifest

Rank	ID	Authoritative file	Primary proof
1	UC-02	04_MVP1_Pilot_Blueprints/UC-02_blueprint.json	Welcome series continuity
2	UC-04	04_MVP1_Pilot_Blueprints/UC-04_blueprint.json	First-purchase incentive delivery
3	UC-05	04_MVP1_Pilot_Blueprints/UC-05_blueprint.json	Double opt-in completion nudge
4	UC-06B	04_MVP1_Pilot_Blueprints/UC-06B_blueprint.json	Second Purchase Accelerator — omnichannel flagship
5	UC-08	04_MVP1_Pilot_Blueprints/UC-08_blueprint.json	Post-purchase education / product usage
6	UC-09	04_MVP1_Pilot_Blueprints/UC-09_blueprint.json	Review & UGC request
7	UC-10	04_MVP1_Pilot_Blueprints/UC-10_blueprint.json	Account creation nudge
8	UC-11	04_MVP1_Pilot_Blueprints/UC-11_blueprint.json	Replenishment reminder
9	UC-12	04_MVP1_Pilot_Blueprints/UC-12_blueprint.json	Back-in-stock notification
10	UC-13	04_MVP1_Pilot_Blueprints/UC-13_blueprint.json	Price-drop alert
11	UC-16	04_MVP1_Pilot_Blueprints/UC-16_blueprint.json	Churn-risk save
12	UC-17	04_MVP1_Pilot_Blueprints/UC-17_blueprint.json	Winback lapsed customers
13	UC-21	04_MVP1_Pilot_Blueprints/UC-21_blueprint.json	Cart abandonment recovery
14	UC-23	04_MVP1_Pilot_Blueprints/UC-23_blueprint.json	VIP candidate progression
15	UC-28	04_MVP1_Pilot_Blueprints/UC-28_blueprint.json	Complementary product cross-sell
16	UC-36	04_MVP1_Pilot_Blueprints/UC-36_blueprint.json	Post-purchase upsell to premium tier

Appendix B — Exact 42-check MVP1 scope

Foundation Gates • RULE_BASED

FD-01 · FD-02 · FD-03 · FD-04 · FD-05 · FD-06 · FD-07

Customer Identity • RULE_BASED

CI-01 · CI-02 · CI-03 · CI-05

Lifecycle Event • RULE_BASED

LE-01 · LE-02 · LE-03 · LE-04 · LE-05 · LE-09

Product & Transaction • RULE_BASED

PT-01 · PT-03 · PT-04

Segment & Property · RULE_BASED

SP-03 · SP-07

Channel & Consent · RULE_BASED

CC-01 · CC-02 · CC-03 · CC-05

Measurement · RULE_BASED

ME-02

Business Reality · RULE_BASED

BR-01

DRIFT / FRESHNESS

CI-13 · CI-14 · CI-15 · LE-08 · LE-11 · LE-13 · PT-14 · SP-08 · SP-12 · CC-12 · ME-08 · ME-09 · BR-02 · BR-12

COUNT ASSERTION 28 RULE_BASED + 14 DRIFT/FRESHNESS = 42. The 102-check catalogue is the expansion library and must not be executed as the MVP1 default scope.

Appendix C — Required acceptance assertions

- All 13 schemas identify JSON Schema Draft 2020-12 and load successfully.
- Exactly sixteen unique pilot blueprints validate and ranks cover 1–16.
- Exactly forty-two check results exist in the Lumera golden run.
- Lumera headline is 84.2670 and state is `CONDITIONALLY_READY`.
- A blocking Foundation Gate suppresses the headline score.
- Required coverage below 80% produces `INCOMPLETE`.
- A failed workflow gating check blocks that workflow regardless of headline DCS.
- No Retire/Consolidate verdict is emitted without complete dependency evidence.
- `DISCOVERY_REQUIRED` never executes without fallback/approval.
- `dependency_cycle.json` produces `BLOCKED_DEPENDENCY_CYCLE` and an explicit path.
- Approval token mismatch produces no side effect.
- Idempotent replay creates no duplicate object or write.
- UC-06B contains persistent holdout, consent branching, global purchase exit and collision policy.
- All blueprint details/tags follow `klints_ / klints:` namespace rules.

- QA hard-fail domains block staging even when average QA score is at least 80.

Appendix D — Operational status and error codes

Code / state	Meaning	Required response
INCOMPLETE	Evidence coverage or sweep insufficient	Collect evidence; do not publish final score/verdict
BLOCKED_CAPABILITY	Required operation unavailable and no fallback	Resolve access, approve fallback or remove from scope
BLOCKED_INPUT	Priority/contract input invalid or missing	Correct source object; do not invent default
BLOCKED_DEPENDENCY_CYCLE	Task graph contains a cycle	Emit cycle path and redesign dependencies
AWAITING_APPROVAL	Side-effect proposal ready but not authorized	Collect valid bound approval; do not execute
IDEMPOTENCY_CONFLICT	Same key used with different payload	Stop and investigate; never overwrite prior meaning
PARTIAL_SWEEP	Connector run incomplete	Checkpoint/resume; no final DCS
SCHEMA_UNSUPPORTED	Object major version not supported	Reject safely and trigger migration/version decision
DATA_GATE_FAILED	Workflow-required DCS check not acceptable	Create remediation task and hold build/activation
MESSAGE_COLLISION	Competing flows target same moment/contact	Apply precedence/suppression; one message per moment

Appendix E — Rohan handoff checklist

- ☐ I verified the ZIP and manifest before importing artifacts.
- ☐ I loaded and versioned all 13 schemas in the backend runtime.
- ☐ I can reproduce the Lumera DCS and every edge-case state.
- ☐ I implemented capability resolution without treating MCP as globally live.
- ☐ I can generate and validate UC-02 and UC-06B build packages.
- ☐ I implemented deterministic task ordering, cycle detection and wave assignment.
- ☐ I implemented approval binding, idempotency and rollback behavior.
- ☐ I ran the complete acceptance matrix and recorded evidence for every pending/failed test.
- ☐ I raised specification issues for every ambiguity instead of resolving them only in code.
- ☐ I completed the final four-role sign-off or documented explicit accepted conditions.

End of implementation blueprint · Klints MVP1 · Version 1.2 · 18 July 2026